

Day-10

DATE:19-08-25

NAME: NANCY M

today's task, add an additional table called users. put api post request to insert user into it in fast api. Additionally, when request to list music api is called, only a valid user is allowed to get the response else raise HTTPException with error code

```
from sqlalchemy import create_engine

from sqlalchemy.dialects.sqlite import *

from sqlalchemy.orm import sessionmaker, Session, declarative_base

from typing import List

from fastapi.templating import Jinja2Templates

from fastapi.responses import HTMLResponse

from sqlalchemy import Column, Integer, String

DATABASE_URL = "sqlite:///./songs.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

class Song(Base):

    __tablename__ = "songs"

    Id = Column(Integer, primary_key=True, index=True)

    Song_Name = Column(String(50), nullable=False)

    Duration = Column(Integer, nullable=False)

class User(Base):
```

```

__tablename__ = "users"

Id = Column(Integer, primary_key=True, index=True)

Name = Column(String(50), unique=True, nullable=False)

Base.metadata.create_all(bind=engine)

from fastapi import FastAPI, HTTPException, Request, Depends

import uvicorn

app = FastAPI()

templates = Jinja2Templates(directory="templates")

def get_db():
    db = SessionLocal()

    try:
        yield db
    finally:
        db.close()

def authenticate(name: str, db: Session):
    user = db.query(User).filter(User.Name == name).first()

    if not user:
        raise HTTPException(status_code=401, detail="User not found")

    return user

from pydantic import BaseModel

class SongSchema(BaseModel):
    Song_Name: str
    Duration: int

```

```

class Config:

    orm_mode = True

class song_dto(BaseModel):

    Name: str

class Config:

    orm_mode = True

class UserSchema(BaseModel):

    Name: str

@app.post("/add_song")

def add_song(s: SongSchema, name: str, db: Session = Depends(get_db)):

    authenticate(name, db)

    st = Song(Song_Name=s.Song_Name, Duration=s.Duration)

    db.add(st)

    db.commit()

    db.refresh(st)

    return st

templates = Jinja2Templates(directory="templates")

@app.get("/list_html", response_class=HTMLResponse)

def get_all_songs_html(request: Request, name: str, db: Session = Depends(get_db)):

    authenticate(name, db)

    recs = db.query(Song).all()

    return templates.TemplateResponse("songlist.html", {"request": request, "songs":
recs})

@app.get("/getBy/{my_id}", response_model=song_dto)

```

```

async def getBy(my_id: int, name: str, db: Session = Depends(get_db)):
    authenticate(name, db)

    st = db.query(Song).filter_by(Id=my_id).first()

    if not st:
        raise HTTPException(status_code=404, detail="Song not found")

    return song_dto(Name=st.Song_Name)

@app.post("/add_user")
def add_user(user: UserSchema, db: Session = Depends(get_db)):
    db_user = db.query(User).filter(User.Name == user.Name).first()

    if db_user:
        raise HTTPException(status_code=400, detail="User already exists")

    new_user = User(Name=user.Name)

    db.add(new_user)

    db.commit()

    db.refresh(new_user)

    return {"message": "User created successfully", "user": new_user.Name}

@app.middleware("http")
async def addmiddleware(request: Request, call_next):
    print("Middleware has intercepted the calls!")

    response = await call_next(request)

    return response

if __name__ == "__main__":
    uvicorn.run("Day10task:app", host="127.0.0.1", port=8000, reload=True)

```

Add song:

Parameters

Cancel

Reset

Name	Description
name * required string (query)	nancy

Request body required

application/json

Edit Value | Schema

```
{
  "Song_Name": "alagiye asura",
  "Duration": 180
}
```

```
curl -X 'POST' \
  'http://127.0.0.1:8000/add_song?name=nancy' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Song_Name": "alagiye asura",
    "Duration": 180
  }'
```

Request URL

http://127.0.0.1:8000/add_song?name=nancy

Server response

Code	Details
200	Response body <pre>{ "Id": 0, "Song_Name": "alagiye asura", "Duration": 180 }</pre> Download Response headers <pre>content-length: 51 content-type: application/json date: Tue, 19 Aug 2025 12:27:50 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Song added to db:

Database Structure			
Browse Data			
Edit Pragmas			
Execute S			
Table: songs			
	<u>Id</u>	Song_Name	Duration
	Filter	Filter	Filter
1	1	anbil avan	180
2	2	ennai konjam matri	240
3	3	Annal Meelae	180
4	4	Vaarayo Vaarayo	300
5	5	visiri	240
6	6	kanima	300
7	7	mumbe va	300
8	8	alagiye	300
9	9	alagiye asura	180

Get list:

default

POST /add_song Add Song

GET /list_html Get All Songs Html

Parameters

Name

Description

name

* required

string

(query)

nancy

Execute

Responses

Code

Description

Links

200

Successful Response

No links

name * required

string

(query)

nancy

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/list_html?name=nancy' \
  -H 'accept: text/html'
```

Request URL

http://127.0.0.1:8000/list_html?name=nancy

Server response

Code

Details

200

Response body

```
<!DOCTYPE html>
<html>
```

Songs list:



Songs List

ID	Song Name	Duration
1	anbil avan	180
2	ennai konjam maatri	240
3	Annal Meelae	180
4	Vaarayo Vaarayo	300
5	visiri	240
6	kanima	300
7	mumbe va	300
8	alagiye	300
9	alagiye asura	180

Getby id:

GET

/list_html1

Get All Songs Html

▼

GET

/getBy/{my_id}

Getby

^

Parameters

Cancel

Name	Description
my_id * required integer (path)	2
name * required string (query)	dharu

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/getBy/2?name=dharu' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/getBy/2?name=dharu
```

Server response

Code	Details
200	Response body <pre>{ "Name": "ennai konjam maatri" }</pre> Download Response headers <pre>content-length: 30 content-type: application/json date: Tue, 19 Aug 2025 12:30:05 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Add user :

GET

/list_html Get All Songs Html

⌵

GET

/getBy/{my_id} Getby

⌵

POST

/add_user Add User

⌴

Parameters

Cancel

Reset

No parameters

Request body required

application/json

⌵

Edit Value

Schema

```
{
  "Name": "jency"
}
```

```
-d '{
  "Name": "jency"
}'
```

Request URL

http://127.0.0.1:8000/add_user

Server response

Code

Details

200

Response body

```
{
  "message": "User created successfully",
  "user": "jency"
}
```

⌵

Download

Response headers

```
content-length: 54
content-type: application/json
date: Tue, 19 Aug 2025 12:31:34 GMT
server: uvicorn
```

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

⌵

Controls Accept header.

Example Value

Schema

User added to db:

Database Structure		Browse Data	
Table:  users			
	<u>Id</u>	Name	
	Filter	Filter	
1	1	nancy	
2	2	dharu	
3	3	nandhini	
4	4	jency	

If we try to access with the unknown name it will shown a user not found:

name * required

string

(query)

asdrftgh

Request body required

application/json

Edit Value | Schema

```
{
  "Song_Name": "kamina",
  "Duration": 0
}
```

Execute

Clear

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/add_song?name=asdrftgh' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Song_Name": "kamima",
    "Duration": 0
  }'
```

Request URL

http://127.0.0.1:8000/add_song?name=asdrftgh

Server response

Code	Details
------	---------

401	Error: Unauthorized
-----	---------------------

Undocumented

Response body

```
{
  "detail": "User not found"
}
```



Download

Response headers

```
content-length: 27
content-type: application/json
date: Tue, 19 Aug 2025 12:32:38 GMT
server: uvicorn
```

Responses